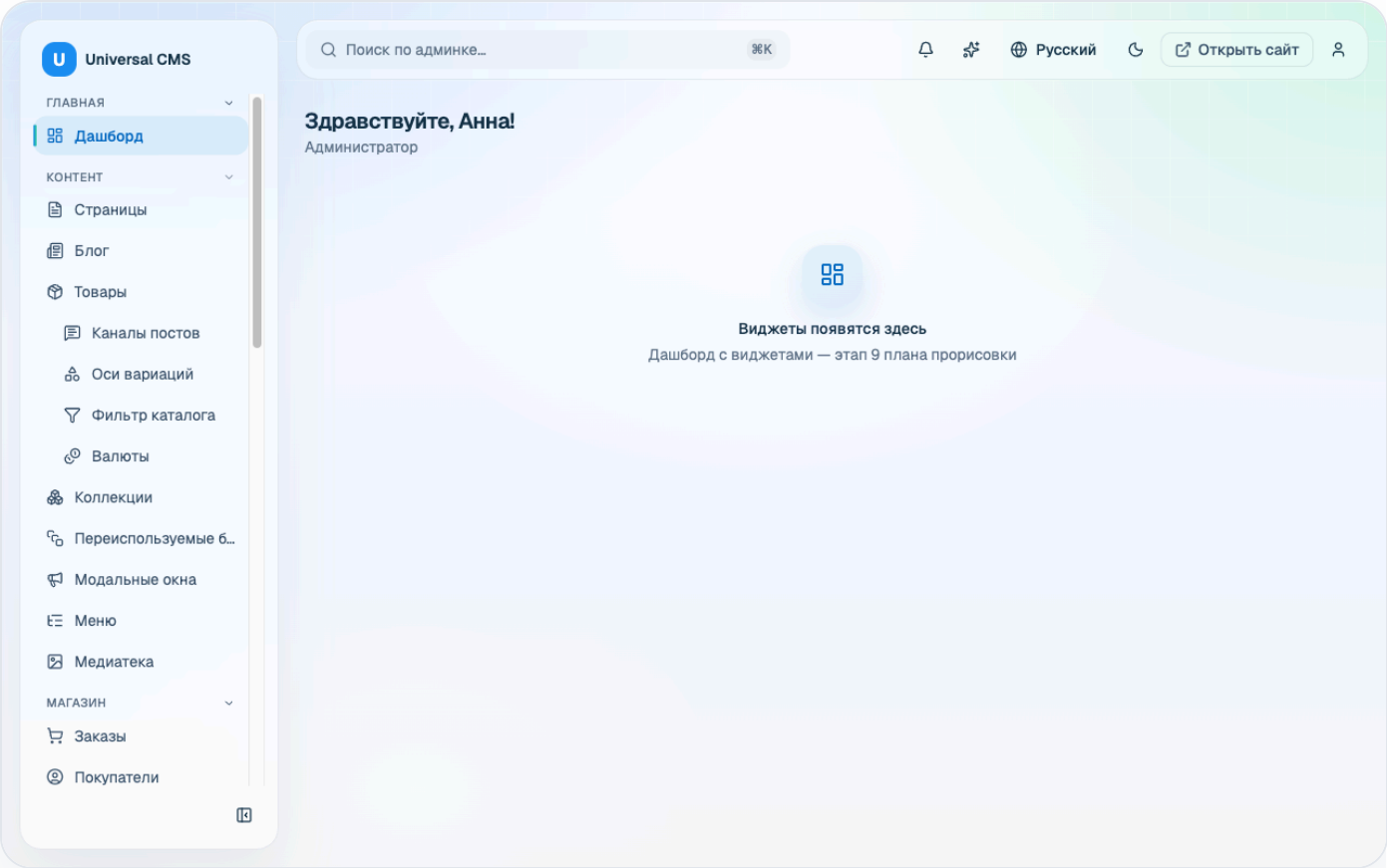
DOCUMENTATION

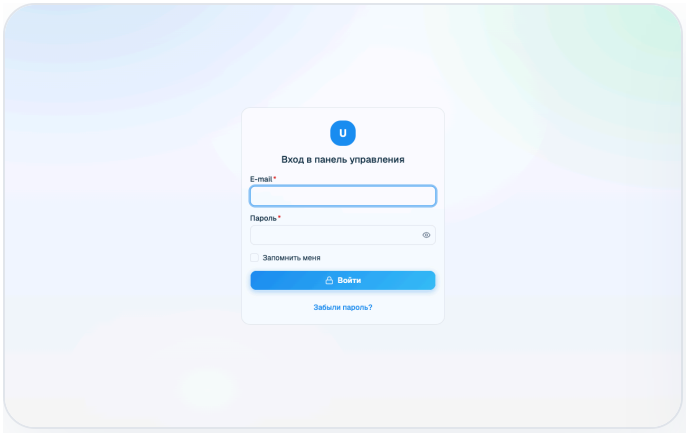
Air Glass UI — Admin Dashboard Template

A premium, production-grade admin dashboard **UI template** built with React 19, TypeScript, Vite and Tailwind CSS v4. It ships with a cohesive “Light Air Glass” design system, **64 accessible UI primitives** plus 67 composed application components, **196 ready-made screens**, three appearance styles, five shell layouts, **9 languages including right-to-left Arabic**, and an in-repo **mock API** so the whole template runs and demos with **zero backend**.

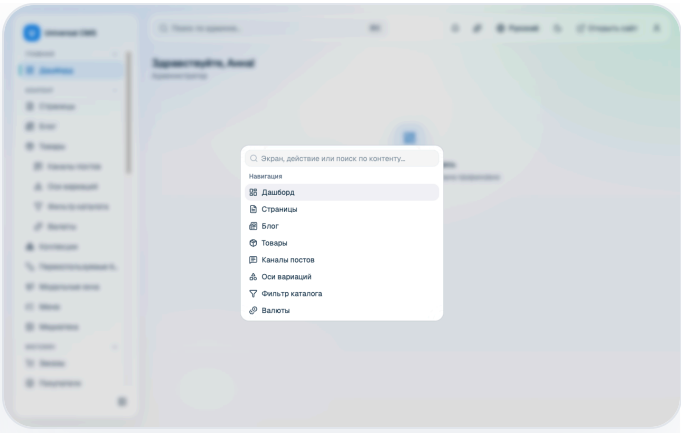
Thank you for purchasing **Air Glass UI**. This document walks you through everything you need to run, customize and extend the template — from a first `npm install` to adding your own screens and swapping the mock data layer for a real backend. Use the sidebar to jump to any topic; the same content prints as a single linear document (see Save as PDF).



The customizable widget dashboard — one of many ready-made screens.



Authentication



⌘K command palette

What's included

Full source code

React 19 + TypeScript sources under `src/`, built with Vite 8.

Design system

“Light Air Glass” tokens centralized in `src/index.css` — light & dark.

64 UI primitives

Accessible primitives in `src/components/ui` (shadcn/ui + Radix + Base UI), plus 67 composed components.

196 ready-made screens

10 dashboards, app suites (e-commerce, CRM, projects, email, support), mono-niches (crypto, NFT, jobs), auth & utility pages, and a 78-page component showcase.

Mock API

A typed client + fixtures so every screen runs and demos without a server.

Appearance & layouts

Three styles (`glass` / `liquid` / `flat`), five shell layouts and a live Theme Customizer drawer.

9 languages, RTL-ready

Nine dictionaries at exact key parity, including Arabic with full right-to-left mirroring.

This documentation

Self-contained HTML you can open offline, plus a print-to-PDF export.

Technology stack

AREA	TECHNOLOGY
Language	TypeScript (strict, target ES2023)
UI runtime	React 19
Build tool	Vite 8 (`@vitejs/plugin-react`)
Styling	Tailwind CSS v4 (`@tailwindcss/vite`), `tw-animate-css` , Geist Variable font
Component primitives	shadcn/ui, Radix UI, Base UI (`@base-ui/react`)
Routing	react-router v8 (browser router, lazy routes)
Data layer	TanStack Query (mock-backed), TanStack Table
Forms & validation	react-hook-form + Zod (`@hookform/resolvers`)
Charts	Recharts
Maps	Leaflet (used directly — no React wrapper), OpenStreetMap tiles

AREA	TECHNOLOGY
Editors	TipTap (rich text), CodeMirror (HTML)
Drag & drop	dnd-kit
Dates	date-fns (locale-aware formatting)
Tooling	oxlint (lint), Prettier (format), Vitest + Testing Library + jsdom (tests), a logical-property RTL guard, and a release packager
Backend	None — in-repo mock API only (no database)

Browser support

Air Glass UI targets modern evergreen browsers — the latest two versions of **Chrome, Edge, Firefox and Safari**. The glass surfaces use `backdrop-filter`; where a browser does not support it the layout degrades gracefully to solid surfaces. There is no Internet Explorer support.

License note. Air Glass UI is distributed under the Envato Market license you purchased (Regular or Extended). Before publishing, review the licensing of every bundled dependency, font and asset for redistribution — the *Geist Variable* font and all runtime dependencies are open-source, but you are responsible for compliance in your own build. The project's `themeforest-prep` notes describe the packaging and licensing bar in detail.

Getting Started

The template is a standard Vite single-page application. If you have built a React app before, everything here will be familiar.

Prerequisites

- **Node.js 20.19+ or 22.12+** (an active LTS release is recommended). Vite 8 requires one of these minimum versions — older Node releases are not supported.
- **npm** (bundled with Node). The commands below use npm; the equivalent `pnpm` or `yarn` commands work too if you prefer, but the lockfile in this package is `package-lock.json` (npm).

Install & run

From the project root, install dependencies and start the dev server:

```
npm install      # install dependencies
npm run dev      # start the Vite dev server with hot-module reloading
```

The dev server prints a local URL (typically `http://localhost:5173/admin-assets/`). Open it and you are running the full template on mock data — no backend required.

> **Why `/admin-assets/`?** The app is configured with a Vite `base` of `/admin-assets/` (see `vite.config.ts`) because it was designed to be mounted under a sub-path. Change `base` to `'/'` if you want to serve it from the domain root. The router reads this value from `import.meta.env.BASE_URL`, so routing follows the base automatically.

Available scripts

All scripts are defined in `package.json`:

COMMAND	WHAT IT DOES
`npm run dev`	Start the Vite dev server with HMR.

COMMAND	WHAT IT DOES
`npm run build`	Type-check then build for production (`tsc -b && vite build`). Output goes to `dist/`.
`npm run preview`	Serve the production build locally to verify it.
`npm run lint`	Lint the codebase with oxlint.
`npm run format`	Format sources with Prettier (`src/**/*.{ts,tsx,css}`).
`npm run format:check`	Check formatting without writing changes.
`npm run test`	Run the test suite once with Vitest.
`npm run lint:rtl`	Fail on physical directional utilities (`ml-*`, `text-left`, …) that would break right-to-left layouts. See RTL & Arabic.
`npm run lint:rtl:all`	The same guard across every file rather than the changed set.
`npm run package`	Run the full quality gate and, only if everything is green, assemble the distributable archive.

Building for production

```
npm run build      # type-checks, then emits static assets into dist/
npm run preview    # locally preview the dist/ build
```

The contents of `dist/` are plain static files — HTML, JS and CSS. Deploy them to any static host (Netlify, Vercel, S3/CloudFront, Nginx, GitHub Pages, …). Because the app is an SPA, configure your host to rewrite unknown paths to `index.html` so client-side routing works on deep links.

Project structure

The source tree follows a **feature-sliced** layout:

```
src/
├── app/              App shell, router and the single navigation map
│   ├── app-shell.tsx    Authenticated layout: sidebar, header, ⌘K palette
│   ├── router.tsx       react-router config; feature screens are lazy-loaded
│   └── nav.ts           ONE navigation map (sidebar + command palette)
├── features/         Screen-level code, one folder per domain (31 slices)
│   ├── auth/            login · signup · forgot · reset · MFA · lock · guest
layout
│   ├── dashboard/       customizable widget dashboard
│   ├── dashboards/      9 vertical dashboards (CRM, ecommerce, crypto, NFT,
…)
│   ├── users/           users list · editor · roles/RBAC · profile
│   ├── settings/        settings + appearance
│   ├── shop/            orders, products, customers, payments, invoices, …
│   ├── crm/ projects/ tasks/ calendar/ email/ support/ todo/   app suites
│   ├── crypto/ nft/ jobs/                                      mono-niches
│   ├── blog/ pages-extra/ landing/ pricing/                    content &
marketing
│   ├── analytics/ inbox/ kanban/ media/ ai/ help/ activity/ …
│   ├── layouts/         shell-layout demo screens
│   └── ui-kit/          78-page component showcase
├── components/
│   ├── ui/              64 reusable primitives (shadcn/ui + Radix + Base UI)
│   ├── charts/ board/   chart compositions and the kanban board
│   └── *.tsx            composed app widgets (data-table, page-header, …)
├── lib/              Cross-cutting helpers (auth, i18n, query, permissions,
…)
├── hooks/            Shared React hooks
├── api/              API client + types
│   └── mock/            in-memory fixtures (zero-backend demo)
├── locales/          9 locale dictionaries (ar, de, en, es, fr, it, pl, ru,
uk)
├── assets/           Static assets
├── index.css         ALL visual identity: shadcn vars + --glass-* tokens
└── main.tsx          App entry: providers + RouterProvider
```

Root configuration files worth knowing:

FILE	PURPOSE
`vite.config.ts`	Vite config: React + Tailwind plugins, `base`, the `@` path alias.
`tsconfig.json` / `tsconfig.app.json` / `tsconfig.node.json`	TypeScript project references (strict mode).
`components.json`	shadcn/ui registry config (points its CSS at `src/index.css`).
`vitest.config.ts`	Vitest test runner config (jsdom environment).
`index.html`	The Vite HTML entry that mounts `main.tsx`.

The `@/*` path alias

Every import that starts with `@/` resolves to `src/`. This is configured in two places that must stay in sync: `resolve.alias` in `vite.config.ts` (for the bundler) and `compilerOptions.paths` in `tsconfig.app.json` (for TypeScript).

```
// Instead of brittle relative paths…
import { Button } from '../../../components/ui/button'

// …use the alias:
import { Button } from '@/components/ui/button'
```

Theming & Design Tokens

Air Glass UI has a **single source of visual truth**: `src/index.css`. Every color, radius, blur, gradient, shadow and glass surface is defined there as a CSS custom property (design token). Screens and components **consume** tokens — they never hard-code their own colors, blur or mesh. This is an enforced project rule, and it is what lets you rebrand the entire template by editing one file.

> **Rule of thumb.** If you find yourself typing a hex color, a `blur()` value, or a gradient inside a component, stop — add or reuse a token in `src/index.css` instead. Keeping tokens centralized is what preserves the "Light Air Glass" consistency across light and dark themes.

How the tokens are organized

`src/index.css` is layered top to bottom:

- **Imports** — Tailwind v4, `tw-animate-css`, the shadcn Tailwind preset, and the Geist Variable font.
- `@theme inline` — maps Tailwind color utilities (`bg-primary`, `text-muted-foreground`, …) to the CSS variables, and defines the radius scale.
- `:root` — the **light** palette: the shadcn variables (`--background`, `--foreground`, `--primary`, `--card`, `--border`, `--radius`, chart colors, sidebar colors) *and* the `--glass-*` layer (glass backgrounds, blur radii, saturation, borders, highlights, shadows), plus motion, glow and status tokens.
- `.dark` — the **dark** overrides: the same token names re-declared on a dark base, so dark mode is a set of value swaps, not a separate stylesheet.

The shadcn palette & the glass layer

Standard shadcn tokens drive component color; the bespoke `--glass-*` tokens drive the frosted-glass surfaces that give the template its identity. A representative slice of the light theme:

```
:root {
  /* shadcn palette (light) */
  --background: #f6faff;
  --foreground: #15324a;
  --primary:    #176dbd;   /* accent — buttons, links, rings, nav marker */
  --card:       var(--glass-bg-card);
  --border:     rgba(148, 163, 184, 0.20);
  --radius:     12px;
  /* charts */
  --chart-1: #1d8df2;  --chart-2: #34d399;  /* … */

  /* --glass-* layer (consumed only by framework surfaces) */
  --glass-bg-card:    rgba(255, 255, 255, 0.42);
  --glass-bg-panel:   rgba(255, 255, 255, 0.50);
  --glass-blur-card:  28px;
  --glass-saturate:   160%;
  --glass-border:     rgba(148, 163, 184, 0.26);
  --glass-shadow:     0 22px 48px rgba(148, 163, 184, 0.16);
}

.dark {
  /* same token names, dark values */
  --background: #0b1220;
  --foreground: #e2ecf7;
  --primary:    #176dbd;
  /* …glass recipe re-applied on a dark base… */
}
```

Note how `--card` points at `--glass-bg-card`: the accent and neutral families *derive* from a small number of base tokens, so changing one value cascades everywhere it is referenced.

How to rebrand

To restyle the whole template, change a handful of token values in `:root` (and the matching ones in `.dark`). The most impactful is `--primary`, the accent that flows into buttons, links, focus rings, the active-nav marker and select highlights:

```
/* src/index.css — before */
:root { --primary: #176dbd; }   /* blue */
.dark { --primary: #176dbd; }

/* after — a violet brand */
:root { --primary: #7c3aed; }   /* buttons, links, rings, nav marker all
follow */
.dark { --primary: #8b5cf6; }   /* a slightly lighter accent reads better on
dark */
```

Save the file and the dev server hot-reloads — every accented surface across every screen updates at once, in both themes, with no per-component edits. To go further, adjust the `--glass-bg-*` opacity for more or less translucency, the `--glass-blur-*` radii for a softer or crisper frost, or the `--radius*` scale for sharper or rounder geometry.

Keep light and dark in step. Every token you change in `:root` should have a considered counterpart in `.dark`. Also check contrast: several tokens carry a note that they were tuned to meet WCAG AA — keep text/background pairs at 4.5:1 or better when you re-color.

Where Tailwind utilities come from

Tailwind CSS v4 is wired in through the `@tailwindcss/vite` plugin (see `vite.config.ts`) and the `@import "tailwindcss";` at the top of `src/index.css` — there is no separate `tailwind.config.js`. Utilities such as `bg-primary` or `rounded-lg` resolve to your tokens because of the `@theme inline` block that maps `--color-*` and `--radius-*` to the CSS variables. Animations come from `tw-animate-css`; the UI font is *Geist Variable* (`@fontsource-variable/geist`).

Appearance, Customizer & Layouts

Beyond light and dark, Air Glass UI ships three **appearance styles** and five **shell layouts**. Both are configuration, not forks: a style is a recipe of token values in `src/index.css`, and a layout re-chromes the same app shell. Nothing is a separate stylesheet or a duplicated component tree, so anything you build inherits all of them for free.

The three appearance styles

Declared in `src/lib/appearance.ts` as `APPEARANCE_STYLES`. Each style only re-declares the `--glass-*` tokens and `--radius`, which is why a component that consumes tokens adapts to all three automatically:

STYLE	BLUR	RADIUS	SATURATION	CHARACTER
`glass` default	28px	12px	160%	The signature “Light Air Glass” frost — translucent surfaces over the mesh background.
`liquid`	40px	18px	190%	A heavier, iOS-like frost: softer geometry, deeper blur, richer saturation.
`flat`	0	10px	100%	No blur at all — opaque surfaces that separate on borders and background alone. The fastest to paint.

> **Verify new work in `flat` too.** `flat` has no blur, so any separation you were unconsciously getting from the frost disappears. If a panel is legible in `flat`, it is legible everywhere. Never hard-code a blur or radius value in a component — read `--glass-blur-*` and `--radius`, and the style switch does the rest.

The five shell layouts

Declared as `APPEARANCE_LAYOUTS`. They change where the navigation lives, not what it contains — all five read the same map from `src/app/nav.ts`:

LAYOUT	SHELL ARRANGEMENT
`vertical` default	The classic left sidebar, collapsible to icons.
`horizontal`	A top menu bar with dropdowns; no sidebar.
`detached`	The sidebar floats away from the viewport edge as a separate glass card.
`two-column`	A narrow icon rail plus a contextual second column for the active section.
`hovered`	An icon rail that expands to full width on hover.

Each layout also has a demo screen under **Pages › Layouts** that switches the site-wide layout on mount, so you can walk all five without touching settings.

The Theme Customizer

A floating drawer (`src/components/theme-customizer.tsx`) reachable from every screen. It writes to the same appearance store the settings screen uses, previews changes live on `<html>` while it is open, and reverts the preview if you close it without saving:

- **Accent** — presets plus a native color input. The whole accent family derives from `--primary` via `color-mix`, so one value re-tints buttons, links, focus rings and the nav marker.
- **Design style** — `glass` / `liquid` / `flat`. Switching adopts that style's token baseline while keeping your accent.
- **Theme** — light or dark.
- **Content width** — fluid or boxed.
- **Layout** — the five shell arrangements above.
- **Direction** — LTR or RTL, applied to `<html dir>` immediately.
- **Density** — comfortable or compact control sizing.
- **Reset** — back to the shipped defaults.
- **Copy config** — copies the current configuration to the clipboard so you can paste it in as your new defaults.

> **Making a customizer choice permanent.** Use **Copy config**, then paste the values into your appearance defaults. The drawer is a preview-and-persist tool for the operator; the shipped defaults are yours to set in source.

RTL & Arabic

The template mirrors properly for right-to-left languages, and ships Arabic (`ar.json`) as a fully translated ninth locale to prove it. Direction is a first-class setting rather than an afterthought.

How direction is toggled

Direction lives in the appearance store and is reflected onto `<html dir>`. Change it from the Theme Customizer's **Direction** control, or from the appearance settings screen. Every layout, spacing and component affordance follows, and direction-implying icons (chevrons, arrows) flip with it.

The rule you must follow when extending

RTL only keeps working if new markup uses Tailwind's **logical** directional utilities instead of the physical ones. This is enforced, not merely recommended:

USE THIS	NEVER THIS
`ms-*` / `me-*`	`ml-*` / `mr-*`
`ps-*` / `pe-*`	`pl-*` / `pr-*`
`start-*` / `end-*`	`left-*` / `right-*`
`text-start` / `text-end`	`text-left` / `text-right`
`border-s` / `border-e`	`border-l` / `border-r`
`rounded-s-*` / `rounded-e-*`	`rounded-l-*` / `rounded-r-*`

```
npm run lint:rtl        # guard the files you changed
npm run lint:rtl:all    # the same guard across the whole codebase
```

The guard (`scripts/check-rtl.mjs`) fails on any physical utility it finds. Run it before you commit; a layout that mirrors correctly in Arabic is a layout that stayed logical.

Adding another RTL language. Follow the add a locale recipe, then mark the locale right-to-left so the direction switches with it. Because the layout rules are already logical, a new RTL language is a dictionary and a flag — not a second set of styles.

Navigation & Information Architecture

`src/app/nav.ts` is the **single navigation map**. The sidebar and the ⌘K command palette both consume it — there is no second list to keep in step, and adding an entry there is what makes a screen appear in both surfaces at once. Parents nest arbitrarily deep (the shipped map goes three levels), and only leaf entries carry a route.

The four top-level sections

SECTION	WHAT BELONGS HERE
Menu	The product itself — dashboards and the application suites (e-commerce, CRM, projects, tasks, calendar, email, support, to-do) plus the mono-niche verticals.
Pages	Standalone pages: authentication variants, utility and error pages, blog, landing pages and the shell-layout demos.
Components	The UI showcase — one page per component family, doubling as living component documentation.
Admin	Air Glass configuration and maintenance: users, roles, settings, appearance, activity log, AI settings, help.

Anatomy of an entry

```
// A leaf — carries a route, and optionally a permission
{ to: "/shop/orders", label: t("nav.orders"), icon: ShoppingCart, perm:
"orders.view" }

// A parent — a collapsible toggle with no route of its own
{
  key: "menu.ecommerce",          // unique; persists the open/closed state
  label: t("nav.ecommerce"),
  icon: Store,
  children: [ /* leaves or further parents */ ],
}
```

Labels are always `t()` keys, never literal strings, so the whole menu translates with the UI. A leaf with a `perm` is hidden from users who lack that permission — in the sidebar *and* in the palette, because both read the same filtered map.

Keep the map honest. Run `node scripts/build-screen-reference.mjs` to print the whole tree with routes and permissions, and to cross-check it against `src/app/router.tsx`. It reports drift in both directions — a menu entry pointing at a route that does not exist, and a route no menu entry can reach — so a broken link surfaces as a report line rather than a 404 a buyer finds. Add `--json` for machine-readable output.

Adding a New Screen

Screens follow a consistent, feature-sliced recipe. Adding one touches six places; do them all and your screen appears in the sidebar and the ⌘K palette, is code-split, permission-gated, localized in all languages, and backed by mock data — exactly like the built-in screens.

1. **Create the feature file** under `src/features/<domain>/`. Export a named React component (PascalCase) and reuse the shared page/layout components (`PageHeader`, `ListLayout`, `DataTable`, `Card`, …) so it matches the rest of the app. Consume design tokens — never hard-code colors.

2. **Register a single navigation entry** in `src/app/nav.ts`. This one map is consumed by *both* the sidebar and the ⌘K command palette — add the entry once and it shows up in both, with no drift. Give it a `perm` if the screen should be gated.

3. **Add a lazy route** in `src/app/router.tsx`. Feature screens load lazily (code-splitting) so the login screen stays tiny; wrap it in `<RequirePermission>` if it needs a permission.

4. **Add i18n keys** to *all 8* locale files under `src/locales/` (see the Internationalization guide and the `i18n-merge.mjs` helper).

5. **Add mock data** under `src/api/mock/` and (optionally) an endpoint on the `api` object so the screen has something to render with zero backend (see Mock API).

6. **Gate with a permission** where appropriate — declare it in the nav entry and in the route, and make sure the mock `me` user carries that permission so you can see the screen.

> **Shortcut.** This project ships an `air-glass-screen` generator skill that scaffolds all six touch-points for you. The steps below show what it produces so you can also do it by hand.

Worked example — a minimal “Announcements” list screen

1. The feature component

```
// src/features/announcements/announcements-page.tsx
import { useQuery } from '@tanstack/react-query'
import { PageHeader } from '@/components/page-header'
import { Card } from '@/components/ui/card'
import { api } from '@/api'
import { t } from '@/lib/i18n'

export function AnnouncementsPage() {
  const { data = [] } = useQuery({
    queryKey: ['announcements'],
    queryFn: () => api.announcements.list(),
  })

  return (
    <div>
      <PageHeader title={t('nav.announcements')} />
      <div className="grid gap-3">
        {data.map((a) => (
          <Card key={a.id} className="p-4">
            <h3 className="font-medium">{a.title}</h3>
            <p className="text-muted-foreground text-sm">{a.body}</p>
          </Card>
        ))}
      </div>
    </div>
  )
}
```

2. One navigation entry (`src/app/nav.ts`)

```
import { Megaphone } from 'lucide-react'
// …inside the appropriate NavGroup's `items` array:
{
  to: '/announcements',
  label: t('nav.announcements'),
  icon: Megaphone,
  perm: 'announcements.view',   // omit `perm` for an ungated screen
},
```

3. A lazy route (`src/app/router.tsx`)

```
const announcementsPage = () =>
  lazyPage(() =>
    import('@/features/announcements/announcements-page').then((m) => ({
      default: m.AnnouncementsPage,
    })),
  )

// …inside the authenticated route tree's `children`:
{
  path: '/announcements',
  element: (
    <RequirePermission perm="announcements.view">
      {announcementsPage()}
    </RequirePermission>
  ),
},
```

4. i18n keys in every locale

```
// Author them once in a payload and merge into all 8 dictionaries:
//   node scripts/i18n-merge.mjs payload.json
// payload.json → { "en": { "nav.announcements": "Announcements" },
//                  "ru": { "nav.announcements": "Объявления" }, … }
```

5. Mock data (`src/api/mock/`) + endpoint

```
// src/api/mock/announcements.ts
export const announcements = [
  { id: 1, title: 'Welcome', body: 'The template is running on mock data.' },
]

// Register it in the mock router (src/api/mock/index.ts) for GET
/announcements,
// and add the typed endpoint on the `api` object in src/api/index.ts:
//   announcements: { list: () => apiFetch<Announcement[]>('/announcements') }
```

Reuse the shared building blocks. For a consistent look, lean on the composed components in `src/components/` — `PageHeader`, `ListLayout`, `SettingsLayout`, `DataTable`, `SaveBar`, `EmptyState`, `PaginationBar` — rather than laying out pages from scratch. The Screen Reference shows how the built-in screens are assembled.

Mock API & Swapping in a Real Backend

Air Glass UI runs end-to-end with **no server**. A small data layer under `src/api/` sits between the screens and the network, and in development it transparently serves responses from in-memory fixtures. Screens cannot tell the difference — by design.

The data layer

FILE	RESPONSIBILITY
`src/api/client.ts`	The transport: `apiFetch()` / `apiStream()`. Handles JSON, query params, CSRF, and cross-cutting behavior (401 → re-login, 403/422/409 error types). Chooses mock vs. real.
`src/api/index.ts`	The typed API surface: the `api` object whose methods (e.g. `api.users.list()`) every screen calls. This is the contract.
`src/api/types.ts`	All request/response TypeScript types.
`src/api/mock/*`	In-memory fixtures per domain (ai, analytics, appearance, dashboard, data, help, inbox, kanban, media, roles, settings, shop, users) plus the mock router in `index.ts`.

How screens consume it (TanStack Query)

Screens never call `fetch` directly. They call an `api.*` method through TanStack Query, which handles caching, loading and error states:

```
import { useQuery } from '@tanstack/react-query'
import { api } from '@/api'

const { data, isLoading } = useQuery({
  queryKey: ['users', filters],
  queryFn: () => api.users.list(filters),
})
```

How the mock switch works

The client decides between mock and real in a single line (`src/api/client.ts`):

```
const useMock =
  import.meta.env.VITE_DEMO === '1' ||
  (import.meta.env.DEV && !import.meta.env.VITE_API_REAL)

const API_BASE = import.meta.env.VITE_API_BASE ?? '/console/api'
```

So by default, in **development** the mock layer is used; in a **production build** (`import.meta.env.DEV` is false) real HTTP is used. You can force real requests during development by setting `VITE_API_REAL`.

Deploying a demo? Build with `VITE_DEMO=1`. A plain `npm run build` deliberately leaves the mock layer *out* of the bundle — that is what you want once you have a real backend, because it keeps ~200 KB of fixtures out of your production build. But it also means the resulting `dist/` has nothing to answer `/console/api/*`: every screen 404s and the app bounces to the login page. To publish a working, backend-free demo, build it explicitly:

```
VITE_DEMO=1 npm run build   # mock bundled — runs standalone
```

The `dist/` included in your download was built this way, so you can host it as-is.

Swapping in a real backend

Because every screen already talks to the typed `api` surface, you have two clean options:

1. **Point the existing client at your server (fastest).** Make your backend speak the same routes and JSON shapes the mock uses, then configure the base URL and disable the mock:

   ```
   # .env.local
   VITE_API_REAL=1                        # use real HTTP even in dev
   VITE_API_BASE=https://api.example.com # your backend base URL
   ```

   With `VITE_API_REAL` set, `apiFetch` issues real `fetch` calls to `API_BASE + path` — no screen or query changes needed.

2. **Rewrite the endpoints (most control).** Keep the shape of the `api` object in `src/api/index.ts` but change the implementations to call your own HTTP (fetch/axios) or adjust paths. Screens and queries import from `@/api` and depend only on the method signatures, so they keep working unchanged.

> **Keep the interface stable.** The golden rule when going live: preserve the *method signatures and return types* of the `api` object. As long as `api.users.list()` resolves to the same `Paginated<UserListItem>`, the UI does not care whether the data came from a fixture or your database. Once you no longer need the demo data, you can delete `src/api/mock/`.

Auth & permissions

Authentication and RBAC in the template are **client-side against the mock**: the current user and their permission set come from `GET /me` (see `api.me()`), the `<AuthProvider>` exposes them, and `<RequirePermission>` / the nav `perm` fields gate routes and menu items. The client already implements the real-world cross-cutting flows — `401` mid-session parks the request and opens a re-login dialog, then retries; `403` surfaces as an error toast; `422` becomes a per-field `ValidationError`; `409` a `ConflictError`; and a `X-CSRF-Token` header (from the `XSRF-TOKEN` cookie) is sent on every mutation. When you connect a real backend, implement those status codes server-side and this behavior keeps working. **Enforce every permission on the server too** — client-side gating is for UX, not security.

Internationalization (i18n)

The admin UI ships in **9 languages**, one of them right-to-left. The system is deliberately lightweight: flat JSON dictionaries bundled into the app and a tiny `t()` helper. UI language is a per-user preference, independent from any content locales. Dictionaries load **on demand** — only the active locale is fetched, so eight unused languages never reach the initial bundle.

Shipped locales

Dictionaries live in `src/locales/`:

FILE	LANGUAGE	FILE	LANGUAGE
`en.json`	English canonical	`it.json`	Italiano
`ru.json`	Русский	`pl.json`	Polski
`uk.json`	Українська	`es.json`	Español
`de.json`	Deutsch	`fr.json`	Français
`ar.json`	العربية RTL	See RTL & Arabic	

`en.json` is the **reference dictionary**: `t()` falls back to the English string when a key is missing in the active locale, and ultimately to the key itself. Keep `en.json` complete.

> **Key parity is enforced.** All nine dictionaries carry the **exact same key set**. The release gate (`npm run package`) fails if any locale gains or loses a key relative to `en.json`, because a half-translated build is a visible defect rather than a silent one. Add every new key to every dictionary — the merge script below does it in one pass.

The `t()` helper

Import `t` from `@/lib/i18n` and translate by key. Hard-coded UI strings are forbidden — always use `t()`. Placeholders use `{name}` syntax:

```
import { t } from '@/lib/i18n'

t('nav.dashboard')                          // "Dashboard"
t('users.greeting', { name: 'Ada' })        // "Welcome, Ada" (from "Welcome,
{name}")
```

The active locale is stored in `localStorage` (`admin.ui_locale`) and defaults to English. The `LocaleSwitcher` component and the `useLocale()` hook read and change it; components re-render on change.

Recipe: add a new key

1. Choose a dotted key, e.g. `nav.reports`.
2. Add it to **all 9** dictionaries with a translated value.
3. Use it in code via `t('nav.reports')`.

To avoid editing nine files by hand, author one payload and run the bundled merge script:

```
// payload.json
{
  "en": { "nav.reports": "Reports" },
  "ru": { "nav.reports": "Отчёты" },
  "uk": { "nav.reports": "Звіти" },
  "de": { "nav.reports": "Berichte" },
  "fr": { "nav.reports": "Rapports" },
  "es": { "nav.reports": "Informes" },
  "it": { "nav.reports": "Report" },
  "pl": { "nav.reports": "Raporty" },
  "ar": { "nav.reports": "التقارير" }
}
```

```
node scripts/i18n-merge.mjs payload.json
# [i18n-merge] en: +1 keys (1 total in payload)
# [i18n-merge] ru: +1 keys …
# …merges the keys into every src/locales/*.json (existing keys are updated)
```

Recipe: add a new locale

1. Copy `src/locales/en.json` to `src/locales/<code>.json` and translate the values.
2. In `src/lib/i18n.ts`: `import` the new file, add its code to the `ADMIN_LOCALES` array, its endonym to `LOCALE_NAMES`, and the import to the `dictionaries` map.
3. Add the new locale key to the `LOCALES` list in `scripts/i18n-merge.mjs` so future merges include it.
4. If the language uses a date format you care about, add its `date-fns` locale to the map in `src/lib/date-locale.ts` — that single map is what every relative and absolute timestamp reads.
5. Run `npm run package` (or just the parity step) to confirm the new dictionary matches `en.json` key for key.

Right-to-left languages. Components are built on Radix / Base UI primitives that respect the document direction, and the appearance store drives `<html dir>`. Arabic ships enabled as a worked example — see RTL & Arabic for the direction toggle and the logical-property rule your own screens must follow.

Component Reference

The template includes **64 UI primitives** in `src/components/ui/`, plus a layer of composed application widgets in `src/components/`. Every primitive is a named export you import from `@/components/ui/<name>`. The interactive **UI-Kit showcase** screen (`/ui-kit`, and the states showcase at `/showcase/states`) is the live demo — open it to see each component with its variants and states.

> **This list stays in sync with the source.** Run `node scripts/build-component-reference.mjs` to enumerate `src/components/ui` directly from disk (add `--json` for machine-readable output). The script prints the grouped catalog and warns about any new file that has not yet been given a documentation entry, so this reference cannot silently drift.

Usage pattern

```
import { Button } from '@/components/ui/button'
import { Card } from '@/components/ui/card'

<Card className="p-4">
  <Button variant="default">Save</Button>
</Card>
```

Forms & inputs 20

IMPORT	PURPOSE
`@/components/ui/button`	Primary action button with variants & sizes.
`@/components/ui/button-group`	Group of segmented / attached buttons.
`@/components/ui/input`	Text input field.
`@/components/ui/input-group`	Input with leading/trailing addons.

IMPORT	PURPOSE
`@/components/ui/input-otp`	One-time-code / OTP input.
`@/components/ui/textarea`	Multi-line text input.
`@/components/ui/label`	Accessible form label.
`@/components/ui/field`	Field wrapper (label + control + description/error).
`@/components/ui/checkbox`	Checkbox control.
`@/components/ui/radio-group`	Mutually-exclusive radio options.
`@/components/ui/switch`	On/off toggle switch.
`@/components/ui/select`	Styled dropdown select (Radix/Base UI).
`@/components/ui/native-select`	Native `<select>` styled to match.
`@/components/ui/combobox`	Autocomplete / searchable select.
`@/components/multi-select`	Multi-value select (lives in `src/components/`, not `ui/`).
`@/components/ui/number-field`	Numeric input with steppers.
`@/components/ui/slider`	Range slider.
`@/components/ui/toggle`	Two-state toggle button.
`@/components/ui/toggle-group`	Group of toggles (single/multiple).
`@/components/ui/calendar`	Date-picker calendar (react-day-picker).
`@/components/ui/rating`	Star rating input/display.

Data display 13

IMPORT	PURPOSE
`@/components/ui/table`	Low-level table primitives.

IMPORT	PURPOSE
`@/components/ui/card`	Glass surface card container.
`@/components/ui/item`	List item row primitive.
`@/components/ui/badge`	Status/label pill with variants.
`@/components/ui/avatar`	User avatar with fallback.
`@/components/ui/aspect-ratio`	Fixed aspect-ratio box.
`@/components/ui/chart`	Recharts chart container + theming.
`@/components/ui/timeline`	Vertical event timeline.
`@/components/ui/stepper`	Multi-step progress indicator.
`@/components/ui/kbd`	Keyboard-shortcut key hint.
`@/components/ui/marker`	Map/point marker element.
`@/components/ui/separator`	Horizontal/vertical divider.
`@/components/ui/progress`	Determinate progress bar.

Overlays & dialogs 11

IMPORT	PURPOSE
`@/components/ui/dialog`	Modal dialog.
`@/components/ui/alert-dialog`	Confirmation dialog with actions.
`@/components/ui/sheet`	Side sheet panel.
`@/components/ui/drawer`	Bottom/side drawer (vaul).
`@/components/ui/popover`	Floating popover.
`@/components/ui/hover-card`	Hover-triggered info card.

IMPORT	PURPOSE
`@/components/ui/tooltip`	Tooltip on hover/focus.
`@/components/ui/dropdown-menu`	Dropdown action menu.
`@/components/ui/context-menu`	Right-click context menu.
`@/components/ui/menubar`	Application menu bar.
`@/components/ui/command`	Command menu (cmdk) — powers the ⌘K palette.

Navigation 11

IMPORT	PURPOSE
`@/components/ui/sidebar`	Collapsible app sidebar primitives.
`@/components/ui/navigation-menu`	Horizontal navigation menu.
`@/components/ui/breadcrumb`	Breadcrumb trail.
`@/components/ui/pagination`	Pagination controls.
`@/components/ui/tabs`	Tabbed sections.
`@/components/ui/accordion`	Expand/collapse accordion.
`@/components/ui/collapsible`	Single collapsible region.
`@/components/ui/scroll-area`	Custom-styled scroll container.
`@/components/ui/resizable`	Resizable split panels.
`@/components/ui/carousel`	Content carousel (embla).
`@/components/ui/direction`	LTR/RTL direction provider.

Feedback 5

IMPORT	PURPOSE
`@/components/ui/alert`	Inline alert banner.
`@/components/ui/sonner`	Toast notifications (sonner).
`@/components/ui/skeleton`	Loading skeleton placeholder.
`@/components/ui/spinner`	Loading spinner.
`@/components/ui/empty`	Empty-state placeholder.

Chat & messaging 4

IMPORT	PURPOSE
`@/components/ui/message`	Chat message bubble container.
`@/components/ui/message-scroller`	Auto-scrolling message list.
`@/components/ui/bubble`	Speech-bubble element.
`@/components/ui/attachment`	File attachment chip/preview.

Composed application widgets

Beyond the primitives, `src/components/` holds higher-level building blocks the screens share — reach for these first when building a page: `PageHeader`, `ListLayout`, `SettingsLayout`, `EditorLayout`, `DataTable`, `DatePicker` / `DateRangePicker`, `SaveBar`, `EmptyState`, `StatusBadge`, `PaginationBar`, `CommandPalette`, `RichTextEditor`, `CodeEditor`, `MediaPicker`, `ConfirmDialog`, `WizardDialog`, `Toast`, `WidgetGrid` and more.

Screen Reference

Screens live under `src/features/*`; their routes are declared in `src/app/router.tsx` and their labels and permissions in `src/app/nav.ts`. The template ships **196 screens**: the **166** reachable from the navigation map, listed below by section, plus **30** authentication, utility and "create" screens that are reached directly or from a list rather than the menu. On top of those sit **17 parameterized detail routes** that render an individual record.

A screen with a permission is wrapped in a `<RequirePermission>` guard and is hidden from both the sidebar and the palette for users who lack it; a dash means any authenticated user can reach it.

> **This table is generated.** Run `node scripts/build-screen-reference.mjs` to reproduce it from `src/app/nav.ts`. The counts here are the script's output, and the release gate re-checks them, so this reference cannot silently drift from the product.

Menu 65

ROUTE	SCREEN	SECTION	PERMISSION
`/`	Default	Dashboards	—
`/analytics`	Analytics	Dashboards	`analytics.view`
`/dashboards/crm`	CRM	Dashboards	`analytics.view`
`/dashboards/ecommerce`	Ecommerce	Dashboards	`analytics.view`
`/dashboards/crypto`	Crypto	Dashboards	`analytics.view`
`/dashboards/projects`	Projects	Dashboards	`analytics.view`
`/dashboards/nft`	NFT	Dashboards	`analytics.view`
`/dashboards/jobs`	Jobs	Dashboards	`analytics.view`
`/dashboards/blog`	Blog	Dashboards	`analytics.view`

ROUTE	SCREEN	SECTION	PERMISSION
`/dashboards/ai`	AI	Dashboards	`analytics.view`
`/inbox`	Chat	Apps	`inbox.view`
`/projects`	Projects	Apps	`projects.view`
`/tasks`	Tasks	Apps	`tasks.view`
`/kanban`	Kanban	Apps	`kanban.view`
`/media`	File Manager	Apps	`media.view`
`/todo`	To Do	Apps	`todo.view`
`/api-keys`	API Key	Apps	`apikeys.view`
`/calendar`	Main calendar	Apps › Calendar	`calendar.view`
`/calendar/month`	Month grid	Apps › Calendar	`calendar.view`
`/email`	Mailbox	Apps › Email	`email.view`
`/email/templates/basic`	Basic	Apps › Email › Templates	`email.view`
`/email/templates/ecommerce`	Ecommerce	Apps › Email › Templates	`email.view`
`/shop/products`	Products	Apps › Ecommerce	`products.view`
`/shop/orders`	Orders	Apps › Ecommerce	`orders.view`
`/shop/cart`	Cart	Apps › Ecommerce	`orders.view`
`/shop/checkout`	Checkout	Apps › Ecommerce	`orders.view`
`/shop/sellers`	Sellers	Apps › Ecommerce	`sellers.view`
`/shop/customers`	Customers	Apps › Ecommerce	`customers.view`

ROUTE	SCREEN	SECTION	PERMISSION
`/shop/invoices`	Invoices	Apps › Ecommerce	`invoices.view`
`/shop/payments`	Payments	Apps › Ecommerce	`payments.view`
`/shop/discounts`	Discounts	Apps › Ecommerce	`orders.discounts`
`/shop/delivery`	Delivery	Apps › Ecommerce	`orders.delivery`
`/crm/contacts`	Contacts	Apps › CRM	`contacts.view`
`/crm/companies`	Companies	Apps › CRM	`crm.companies`
`/crm/deals`	Deals	Apps › CRM	`crm.deals`
`/crm/leads`	Leads	Apps › CRM	`crm.leads`
`/crypto/transactions`	Transactions	Apps › Crypto	`crypto.view`
`/crypto/buy-sell`	Buy & Sell	Apps › Crypto	`crypto.view`
`/crypto/orders`	Orders	Apps › Crypto	`crypto.view`
`/crypto/wallet`	My Wallet	Apps › Crypto	`crypto.view`
`/crypto/ico`	ICO List	Apps › Crypto	`crypto.view`
`/crypto/kyc`	KYC Application	Apps › Crypto	`crypto.view`
`/nft/marketplace`	Marketplace	Apps › NFT Marketplace	`nft.view`
`/nft/explore`	Explore	Apps › NFT Marketplace	`nft.view`
`/nft/auction`	Live Auction	Apps › NFT Marketplace	`nft.view`
`/nft/collections`	Collections	Apps › NFT Marketplace	`nft.view`

ROUTE	SCREEN	SECTION	PERMISSION
`/nft/creators`	Creators	Apps › NFT Marketplace	`nft.view`
`/nft/ranking`	Ranking	Apps › NFT Marketplace	`nft.view`
`/nft/wallet-connect`	Wallet Connect	Apps › NFT Marketplace	`nft.view`
`/nft/create`	Create NFT	Apps › NFT Marketplace	`nft.manage`
`/jobs/statistics`	Statistics	Apps › Jobs	`jobs.view`
`/jobs/list`	Job List	Apps › Jobs	`jobs.view`
`/jobs/grid`	Job Grid	Apps › Jobs	`jobs.view`
`/jobs/candidates`	Candidates	Apps › Jobs	`jobs.view`
`/jobs/candidates/grid`	Candidates Grid	Apps › Jobs	`jobs.view`
`/jobs/application`	Application	Apps › Jobs	`jobs.view`
`/jobs/new`	New Job	Apps › Jobs	`jobs.manage`
`/jobs/companies`	Companies	Apps › Jobs	`jobs.view`
`/jobs/categories`	Categories	Apps › Jobs	`jobs.manage`
`/support/tickets`	Ticket List	Apps › Support Tickets	`support.view`
`/layouts/vertical`	Default	Layouts	—
`/layouts/horizontal`	Horizontal	Layouts	—
`/layouts/detached`	Detached	Layouts	—

ROUTE	SCREEN	SECTION	PERMISSION
`/layouts/two-column`	Two column	Layouts	—
`/layouts/hovered`	Hovered	Layouts	—

Pages 13

ROUTE	SCREEN	SECTION	PERMISSION
`/profile`	Profile	Pages	—
`/pricing`	Pricing	Pages	—
`/starter`	Starter	Utility	—
`/team`	Team	Utility	—
`/timeline`	Timeline	Utility	—
`/faq`	FAQs	Utility	—
`/gallery`	Gallery	Utility	—
`/sitemap`	Sitemap	Utility	—
`/blog/list`	Blog list	Blog	—
`/blog/grid`	Blog grid	Blog	—
`/landing`	One page	Landing	—
`/landing/nft`	NFT	Landing	—
`/landing/job`	Jobs	Landing	—

Components 78

ROUTE	SCREEN	SECTION	PERMISSION
`/components/base/buttons`	Buttons	Base UI	—
`/components/base/alerts`	Alerts	Base UI	—
`/components/base/badges`	Badges	Base UI	—
`/components/base/colors`	Colors	Base UI	—
`/components/base/cards`	Cards	Base UI	—
`/components/base/carousel`	Carousel	Base UI	—
`/components/base/dropdowns`	Dropdowns	Base UI	—
`/components/base/grid`	Grid	Base UI	—
`/components/base/images`	Images	Base UI	—
`/components/base/tabs`	Tabs	Base UI	—
`/components/base/accordion`	Accordion	Base UI	—
`/components/base/modals`	Modals	Base UI	—
`/components/base/offcanvas`	Offcanvas	Base UI	—
`/components/base/placeholders`	Placeholders	Base UI	—
`/components/base/progress`	Progress	Base UI	—
`/components/base/notifications`	Notifications	Base UI	—
`/components/base/media`	Media	Base UI	—
`/components/base/video`	Video	Base UI	—
`/components/base/typography`	Typography	Base UI	—

ROUTE	SCREEN	SECTION	PERMISSION
`/components/base/lists`	Lists	Base UI	—
`/components/base/links`	Links	Base UI	—
`/components/base/general`	General	Base UI	—
`/components/base/ribbons`	Ribbons	Base UI	—
`/components/base/utilities`	Utilities	Base UI	—
`/components/advance/sweet-alerts`	Sweet Alerts	Advance UI	—
`/components/advance/nestable-list`	Nestable List	Advance UI	—
`/components/advance/scrollbar`	Scrollbar	Advance UI	—
`/components/advance/animation`	Animation	Advance UI	—
`/components/advance/tour`	Tour	Advance UI	—
`/components/advance/swiper-slider`	Swiper / Slider	Advance UI	—
`/components/advance/ratings`	Ratings	Advance UI	—
`/components/advance/highlight`	Highlight	Advance UI	—
`/components/advance/scrollspy`	ScrollSpy	Advance UI	—
`/components/widgets`	Widgets	—	—
`/components/icons`	Icons	—	—
`/components/forms/basic-elements`	Basic Elements	Forms	—
`/components/forms/form-select`	Select	Forms	—
`/components/forms/checks-radios`	Checkboxes & Radios	Forms	—
`/components/forms/pickers`	Pickers	Forms	—

ROUTE	SCREEN	SECTION	PERMISSION
`/components/forms/input-masks`	Input Masks	Forms	—
`/components/forms/advanced`	Advanced Inputs	Forms	—
`/components/forms/range-slider`	Range Slider	Forms	—
`/components/forms/validation`	Validation	Forms	—
`/components/forms/wizard`	Wizard	Forms	—
`/components/forms/editors`	Editors	Forms	—
`/components/forms/file-uploads`	File Uploads	Forms	—
`/components/forms/form-layouts`	Form Layouts	Forms	—
`/components/forms/select2`	Select2	Forms	—
`/components/tables/basic`	Basic Tables	Tables	—
`/components/tables/gridjs`	Sortable Table	Tables	—
`/components/tables/listjs`	Searchable List	Tables	—
`/components/tables/datatables`	Data Tables	Tables	—
`/components/charts/line`	Line	Charts	—
`/components/charts/area`	Area	Charts	—
`/components/charts/column`	Column	Charts	—
`/components/charts/bar`	Bar	Charts	—
`/components/charts/mixed`	Mixed	Charts	—
`/components/charts/timeline`	Timeline	Charts	—
`/components/charts/range-area`	Range area	Charts	—

ROUTE	SCREEN	SECTION	PERMISSION
`/components/charts/funnel`	Funnel	Charts	—
`/components/charts/candlestick`	Candlestick	Charts	—
`/components/charts/boxplot`	Box plot	Charts	—
`/components/charts/bubble`	Bubble	Charts	—
`/components/charts/scatter`	Scatter	Charts	—
`/components/charts/heatmap`	Heatmap	Charts	—
`/components/charts/treemap`	Treemap	Charts	—
`/components/charts/pie`	Pie	Charts	—
`/components/charts/radialbar`	Radial Bar	Charts	—
`/components/charts/radar`	Radar	Charts	—
`/components/charts/polar-area`	Polar Area	Charts	—
`/components/charts/slope`	Slope	Charts	—
`/components/charts/chartjs`	Chart.js	Charts	—
`/components/charts/echarts`	ECharts	Charts	—
`/components/maps/base`	Base map	Maps	—
`/components/maps/markers`	Markers & choropleth	Maps	—
`/components/maps/substitution`	Vector / Google → Leaflet	Maps	—
`/components/multi-level/page-1`	Level 1.1	Multi Level › Level 1	—

ROUTE	SCREEN	SECTION	PERMISSION
`/components/multi-level/page-2`	Level 2.1	Multi Level › Level 1 › Level 2	—

Admin 10

ROUTE	SCREEN	SECTION	PERMISSION
`/settings/site`	Settings	Configuration	`settings.manage`
`/appearance`	Appearance	Configuration	`settings.manage`
`/ai`	AI assistant	Configuration	`ai.use`
`/system/ai`	AI settings	Configuration	`ai.manage`
`/users`	Users	Access	`users.view`
`/roles`	Roles	Access	`roles.manage`
`/system/activity`	Activity log	System	`activity.view`
`/help`	Help	System	—
`/ui-kit`	UI Kit	System	—
`/showcase/states`	States	System	—

Authentication & account 18

Guest screens rendered outside the authenticated shell. Every one has a **cover** variant — the same flow with a split hero panel — so you can pick the look that fits your brand.

ROUTE	SCREEN
`/login`	Sign-in (2FA challenge, impersonation return)
`/login/cover`	Sign-in — cover variant

ROUTE	SCREEN
`/signup`	Sign-up
`/signup/cover`	Sign-up — cover variant
`/forgot`	Forgot-password request
`/forgot/cover`	Forgot password — cover variant
`/reset/:token`	Password reset
`/reset/:token/cover`	Password reset — cover variant
`/password-create`	Create a password
`/password-create/cover`	Create a password — cover variant
`/verify`	Verify e-mail / code entry
`/verify/cover`	Verify — cover variant
`/lock`	Lock screen
`/lock/cover`	Lock screen — cover variant
`/logout`	Signed-out confirmation
`/logout/cover`	Signed out — cover variant
`/auth-success`	Success confirmation
`/auth-success/cover`	Success — cover variant

Utility & error pages 10

ROUTE	SCREEN
`/404`	Not found
`/404-alt`	Not found — alternate illustration

ROUTE	SCREEN
`/404-cover`	Not found — cover variant
`/500`	Server error
`/offline`	Offline
`/maintenance`	Maintenance mode
`/coming-soon`	Coming soon (countdown)
`/privacy`	Privacy policy
`/terms`	Terms and conditions
`/search-results`	Search results

Detail & editor routes 19

Reached from their list screen rather than the navigation map:

ROUTE	SCREEN
`/users/new`	Create a user
`/users/:id`	Edit a user
`/projects/new`	Create a project
`/projects/:id`	Project detail
`/tasks/:id`	Task detail
`/shop/products/new`	Create a product
`/shop/products/:id`	Product detail
`/shop/products/:id/edit`	Edit a product
`/shop/orders/:id`	Order detail

ROUTE	SCREEN
`/shop/customers/:id`	Customer detail
`/shop/invoices/new`	Create an invoice
`/shop/invoices/:id`	Invoice detail
`/shop/sellers/:id`	Seller detail
`/support/tickets/:id`	Ticket conversation
`/blog/:id`	Blog article
`/jobs/:id`	Job posting detail
`/nft/item/:id`	NFT item detail
`/help/:module/:page`	Help article
`/settings/:tab?`	Settings tab

Save as PDF

This documentation is designed to print cleanly to a single, linear PDF. A dedicated print stylesheet hides the sidebar and chrome, expands every section, adds page-break hints and switches to print-friendly typography.

The quickest way

1. Open `documentation/index.html` in your browser.
2. Press the **Print / Save as PDF** button at the top (or `Ctrl/Cmd + P`).
3. Choose **"Save as PDF"** as the destination and save it as `documentation.pdf`.

Reproducible command-line export

For a scripted, repeatable export, use headless Chrome. A helper is included at `scripts/build-docs-pdf.mjs`; it shells out to a Chrome/Chromium binary you already have (it adds **no** heavy dependency to `package.json`):

```
# uses $CHROME_BIN, or a common macOS/Linux Chrome/Chromium path
node scripts/build-docs-pdf.mjs

# or point it at your browser explicitly
CHROME_BIN="/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" \
  node scripts/build-docs-pdf.mjs
```

The script renders `documentation/index.html` to `documentation/documentation.pdf`. Equivalent one-liners:

```
# Headless Chrome directly
"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" \
  --headless --disable-gpu --no-pdf-header-footer \
  --print-to-pdf=documentation/documentation.pdf \
  documentation/index.html

# …or wkhtmltopdf, if you have it installed
wkhtmltopdf --enable-local-file-access \
  documentation/index.html documentation/documentation.pdf
```

Tip. In the browser Print dialog, enable *"Background graphics"* for the closest match to the on-screen look, and keep margins at the default — the stylesheet already sets a sensible `@page` margin.